SUMADHURA

Flutter Developer - Assignment

Project	Visitor Management Mobile Application
Position	Flutter Developer
Assignment Type	Practical Mobile Application Development
Technology	Flutter & Dart
Backend	Mock REST API (No Backend Development Required)
Submission Timeline	2 to 3 days

Overview

Develop a **Visitor Management Mobile Application** in Flutter that lets front-desk and security personnel register visitors, manage visitor information, generate and scan QR codes, and record visitor check-in/check-out activities.

The application should demonstrate your practical understanding of Flutter development, REST API integration, CRUD operations, state management, reusable UI components, and application workflows.

Objectives

- Develop a functional Flutter mobile application
- Integrate REST APIs for visitor data management
- Implement Create, Read, Update, and Delete (CRUD) operations
- Implement QR code generation and scanning
- Manage visitor check-in/check-out activities
- Build reusable and responsive UI components
- Demonstrate clean coding and error-handling practices

Scope: This is a frontend-focused assignment. Candidates are not required to develop a separate backend application.

Scenario

A real estate organization manages customer and vendor visits across multiple residential projects.

Front-desk and security personnel need a mobile application to register visitors, identify them using QR codes, and maintain records of their check-in and check-out activities.

The proposed application will help staff:

1. Register new visitors
2. View and manage existing visitor records
3. Generate QR codes for registered visitors
4. Scan QR codes to retrieve visitor information
5. Record visitor entry and exit
6. View visitor activity statistics

The application should be easy to operate, with minimal navigation steps and clear status indicators.

Functional Requirements

1. Dashboard

Create a dashboard displaying the following summary information:

Metric	Description
Total Visitors Today	Visitors registered today
Currently Checked In	Visitors whose current status is Checked In
Checked Out Today	Visitors checked out today

The dashboard should also provide shortcuts to:

- Register Visitor
- Visitor List
- Scan QR Code

Statistics should be calculated from the records retrieved through the API. A separate dashboard API is not required.

Use the device's local date consistently for today's metrics.

2. Visitor Registration

Provide a form for registering new visitors.

Field	Input Type	Validation
Visitor Name	Text	Required
Mobile Number	Numeric	Required, exactly 10 digits
Email	Email	Optional; validate when entered
Visitor Type	Dropdown	Customer, Vendor, Other
Project	Dropdown	Project A, Project B, Project C
Purpose of Visit	Text	Required

Required Functionality

- Register new visitors
- Edit existing visitor information
- Save data through REST APIs
- Display validation messages
- Display confirmation after successful registration
- Generate a QR code using the visitor ID returned by the API

The default status of a newly registered visitor should be `Registered`.

3. Visitor Listing & Management

Display registered visitors in a scrollable list.

Each record should show:

- Visitor name
- Mobile number
- Project name
- Visitor type
- Current status

Required Operations

- Retrieve visitor records from the API
- Search visitors by name or mobile number
- View visitor details

- Edit visitor information
- Delete visitor records after confirmation

Search may be implemented locally using the retrieved visitor records.

4. QR Code Generation & Scanning

Implement QR code functionality using suitable Flutter packages.

QR Code Generation

- Generate a QR code after successful registration
- QR code must contain the unique visitor ID
- Display the QR code on the Visitor Details screen

QR Code Scanning

- Open the device camera
- Scan a visitor QR code
- Extract the visitor ID
- Retrieve the visitor details from the API
- Display the visitor information and status
- Allow check-in or check-out based on the visitor's status

Validation

- Show an error for an invalid QR code
- Show an error if the visitor does not exist
- Prevent repeated scanner callbacks from performing duplicate actions
- Prevent duplicate check-in

5. Visitor Check-in / Check-out

Implement the following status workflow:

`Registered → Checked In → Checked Out`

Check-in

- Allow check-in only when status is `Registered`
- Record check-in date and time
- Update status to `Checked In`
- Save the changes through the API

Check-out

- Allow check-out only when status is `Checked In`
- Record check-out date and time
- Update status to `Checked Out`
- Save changes through the API

Business Rules

- Visitors cannot check out before checking in
- Visitors cannot check in twice
- Completed visits cannot be checked in again
- Visitor listing and dashboard must reflect updated status

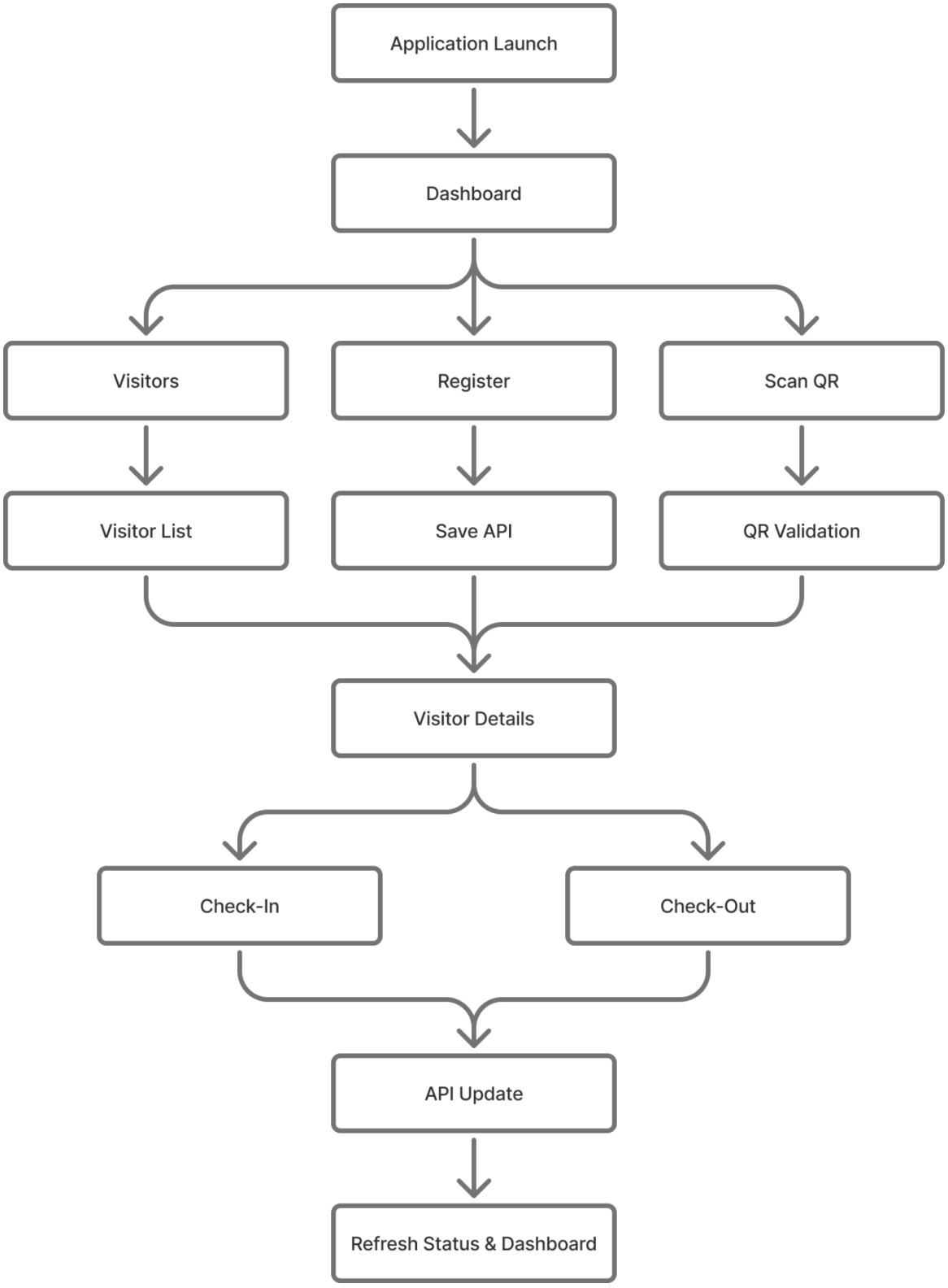
For this assignment, one visit session per visitor registration is sufficient. Repeat visits and visit history management are not required.

Application Screens & Navigation Flow

The application should contain four primary screens, plus a QR scanner view.

Screen	Purpose
Dashboard	Visitor statistics and navigation
Visitor List	Search, view, edit, delete
Visitor Registration / Edit	Capture and update visitor information
Visitor Details	Show visitor profile, QR code, and check-in/check-out
QR Scanner View	Scan QR codes and identify visitors

Suggested Navigation Flow



Candidates may use Flutter Navigator, GoRouter, or an equivalent navigation approach.

UI Components & Design Requirements

The application should have a clean, modern, and user-friendly interface designed for front-desk and security personnel.

Candidates are expected to demonstrate appropriate usage of Flutter widgets, reusable components, responsive layouts, and consistent styling.

a. Mandatory UI Components

UI Component	Suggested Flutter Widget	Expected Usage
App Bar	`AppBar`	Titles, navigation, and actions
Bottom Navigation	`NavigationBar` / `BottomNavigationBar`	Dashboard, Visitors, and Scan
Dashboard Cards	`Card`, `Container`	Visitor statistics
Visitor List	`ListView.builder`	Display visitor records
Visitor Information Card	`Card`, `ListTile`	Visitor summary
Text Input	`TextFormField`	Registration details
Dropdown Selection	`DropdownButtonFormField`	Visitor type and project
Search Field	`SearchBar` / `TextField`	Search visitors
Action Buttons	`ElevatedButton`, `OutlinedButton`	Register, Save, Check-in, Check-out
Icon Buttons	`IconButton`	Edit, delete, scan
Status Indicators	`Chip`, `Badge`	Visitor status
QR Code Display	`qr_flutter`	Generate visitor QR
QR Scanner	`mobile_scanner`	Scan QR using camera
Confirmation Dialog	`AlertDialog`	Delete confirmation
Notifications	`SnackBar`	Success/error messages

Loading Indicator	`CircularProgressIndicator`	API loading
Empty State	`Center`, `Column`, `Icon`	No visitor records

Equivalent widgets or packages may be used.

b. Screen-Wise UI Requirements

Dashboard

- Application title
- Three visitor statistics cards
- Register Visitor action button
- Scan QR action button
- Bottom navigation

Visitor List

- Search field
- Scrollable visitor cards
- Visitor name, mobile, project, and status
- Edit/delete actions
- Tap to open details

Visitor Registration / Edit

- Form text fields
- Dropdown selectors
- Required-field validation
- Save and Cancel buttons
- Scrollable keyboard-friendly layout

Visitor Details

- Visitor information card
- QR code display
- Status indicator
- Check-in/check-out action
- Check-in/check-out timestamps when available

QR Scanner View

- Camera preview
- QR scanning area
- Invalid QR feedback

- Navigation to visitor details after successful scanning

c. Reusable Components

Create reusable UI components where appropriate.

Suggested components:

- `CustomTextField`
- `CustomButton`
- `DashboardStatCard`
- `VisitorCard`
- `StatusChip`
- `LoadingView`
- `EmptyStateWidget`

Exact names and implementations are flexible.

d. Theme & Design Consistency

- Use `ThemeData` for consistent styling
- Follow Material Design principles
- Maintain consistent spacing and typography
- Use suitable icons and status colors
- Keep navigation simple
- Avoid unnecessary animations

e. Responsive Layout

The application should:

- Support common Android screen sizes
- Avoid overflow when the keyboard is displayed
- Use scrollable forms
- Handle safe areas correctly
- Maintain readable layouts on smaller screens

Advanced tablet support is optional.

f. UI State Handling

Provide suitable interfaces for:

- Loading

- Success
- Empty list
- Network failure
- Form validation errors
- Invalid QR codes
- Check-in/check-out failure

The application should not crash or display blank screens during normal error conditions.

Data Models & Sample JSON

Create a visitor data model with the following properties.

1. Visitor Data Model

Field	Data Type	Description
id	String	Unique visitor ID
name	String	Visitor name
mobile	String	Mobile number
email	String/null	Email address
visitor_type	String	Customer / Vendor / Other
project	String	Selected project
purpose	String	Purpose of visit
status	String	Registered / Checked In / Checked Out
created_at	String	Registration timestamp
check_in_time	String/null	Check-in timestamp
check_out_time	String/null	Check-out timestamp

2. Sample Visitor JSON

```
None
{
  "id": "VIS001",
```

```
  "name": "Rahul Sharma",
  "mobile": "9876543210",
  "email": "rahul@example.com",
  "visitor_type": "Customer",
  "project": "Project A",
  "purpose": "Site Visit",
  "status": "Registered",
  "created_at": "2026-10-26T10:00:00+05:30",
  "check_in_time": null,
  "check_out_time": null
}
```

The sample structure is illustrative. Candidates may adapt field names to their chosen mock API, provided all required information is available.

3. Check-in Update Example

```
None
{
  "status": "Checked In",
  "check_in_time": "2026-10-26T10:30:00+05:30"
}
```

4. Check-out Update Example

```
None
{
  "status": "Checked Out",
  "check_out_time": "2026-10-26T11:30:00+05:30"
}
```

The application should deserialize API responses into Dart models and serialize changes for API updates.

REST API Integration

Use a mock REST API service such as MockAPI or an equivalent service supporting persistent CRUD operations.

A separate backend project is not required.

a. Suggested API Endpoints

Method	Endpoint	Functionality
GET	`/visitors`	Retrieve all visitors
GET	`/visitors/{id}`	Retrieve visitor details
POST	`/visitors`	Register visitor
PUT/PATCH	`/visitors/{id}`	Edit visitor
DELETE	`/visitors/{id}`	Delete visitor
PATCH	`/visitors/{id}`	Update visit status and timestamps

The actual endpoints may differ depending on the selected API provider.

b. API Requirements

Candidates must demonstrate:

- Asynchronous API calls
- JSON serialization/deserialization
- GET, POST, PUT/PATCH, and DELETE operations
- Proper handling of HTTP errors
- Loading and success states
- Appropriate user feedback
- Refreshing records after updates
- Handling of unavailable or invalid API responses

c. API Implementation Guidelines

- Avoid embedding API requests directly inside UI widgets
- Use a dedicated API service or repository class
- Keep the API base URL configurable
- Handle unsuccessful network operations without crashing
- Use server-generated visitor IDs for QR generation and lookup

Mock services should persist visitor changes so that CRUD and status updates can be tested after restarting the application.

Technical Requirements & State Management

a. Technology Stack

Component	Requirement
Mobile Framework	Flutter
Language	Dart
Platform	Android
State Management	Provider / Riverpod / BLoC
REST API Client	Dio / `http`
QR Generation	`qr_flutter` or equivalent
QR Scanning	`mobile_scanner` or equivalent
Architecture	Modular, maintainable structure
Version Control	Git

b. State Management

Use an appropriate state management approach to handle:

- Visitor data loading.
- Visitor list updates.
- Registration/editing operations.
- Search results.
- Visitor status changes.
- Dashboard statistics.
- API errors.

For example, when a visitor checks in, the updated status should appear in both the visitor details and listing screens, and dashboard counts should update without restarting the application.

Candidates should briefly explain their choice of state management approach in the README.

c. Device Integration

Use the Android device camera for QR scanning.

Handle camera permissions correctly and display a meaningful message when permission is denied.

Testing on a physical Android device is recommended.

Optional / Bonus Features

The following features are optional and should be attempted only after completing mandatory requirements.

- Filter visitor records by project or status
- Pull-to-refresh visitor listing
- Local data caching
- Visitor check-in/check-out history
- Basic automated unit or widget tests
- Dark mode

Bonus features are not required for successful completion of the assignment.

Expected Deliverables

Candidates must submit the following:

1. Source Code

- Complete Flutter project
- GitHub repository link
- All necessary project configuration files

2. Android APK

Provide an installable Android APK for evaluation.

The application should launch and run without additional development work.

3. README Documentation

Include:

- Project overview

- Flutter/Dart version used
- Setup and execution instructions
- Packages and dependencies
- State management approach
- Mock API details
- Application features
- Known limitations, if any

4. Screenshots

Provide screenshots of:

- Dashboard
- Visitor List
- Visitor Registration
- Visitor Details & QR Code
- QR Scanner

5. API Information

Provide the mock API base URL, sample records, or setup instructions needed to test the application.

The API should remain accessible during evaluation.

Evaluation Considerations

Flutter UI/UX: Proper widget usage, responsive screens, navigation, reusable components, and consistent design.

REST APIs: Working CRUD operations, asynchronous handling, error feedback, and data persistence.

QR Functionality: Correct QR generation and scanning, visitor lookup, and invalid-scan handling.

State Management: Accurate synchronization of visitor data, status, and dashboard counts.

Functional Logic: Correct visitor registration, validation, check-in/check-out, and duplicate-action prevention.

Code Quality: Readable, maintainable, and properly structured code.

Documentation: Clear setup instructions, working APK, and complete submission.

The primary focus is on functionality, code quality, and technical understanding rather than advanced UI effects.

Important Notes

- Backend development is not required
- REST API integration is mandatory
- QR code generation and scanning are mandatory
- Working functionality takes priority over unnecessary features
- Candidates may use Flutter packages and external development references
- AI-assisted tools are not to be used; candidates must understand and be able to explain their implementation
- Candidates should be prepared to demonstrate their application and explain the architecture, API handling, state management, and business logic during the technical interview.

Conclusion & Best Wishes

This assignment is an opportunity to demonstrate your Flutter development skills, technical approach, and problem-solving abilities.

We encourage you to focus on functionality, code quality, and user experience.

All the best! We look forward to reviewing your submission.